R Notebook

Authors: Leonel Fernando Nabaza Ruibal and Michael Jiménez González

Exercise 1: Preprocessing and multivariate analysis

In this first exercise we are going to work with a real datasets (*Palmer Penguins or the Penguin Data*) in order to put into practice the concepts and techniques explained in the theory class about preprocessing.

The work **must** be done in pairs.

```
list.of.packages = c(
  "ggplot2",
  "outliers",
  "readr",
  "mvoutlier",
  "chemometrics",
  "adamethods",
  "DMwR2",
  "dplyr",
  "patchwork",
  "Rlof",
  "solitude",
  "tidyverse",
  "palmerpenguins",
  "missForest",
  "mice"
  )

new.packages <- list.of.packages[!(list.of.packages %in% installed.packages()[,"Package"])]
if(length(new.packages) > 0) {
  install.packages(new.packages)
}
lapply(list.of.packages, require, character.only = T)
```

```
## Cargando paquete requerido: ggplot2

## Cargando paquete requerido: outliers

## Cargando paquete requerido: readr

## Cargando paquete requerido: mvoutlier

## Cargando paquete requerido: sgeostat
```

```
## Cargando paquete requerido: chemometrics

## Cargando paquete requerido: rpart

## Cargando paquete requerido: adamethods

## Cargando paquete requerido: DMwR2

## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo

## Cargando paquete requerido: dplyr

##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

## Cargando paquete requerido: patchwork

## Cargando paquete requerido: Rlof

## Cargando paquete requerido: doParallel

## Cargando paquete requerido: foreach

## Cargando paquete requerido: iterators

## Cargando paquete requerido: parallel

## Cargando paquete requerido: solitude

## Cargando paquete requerido: tidyverse

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.1     v stringr   1.5.2
## v lubridate 1.9.4     v tibble    3.3.0
## v purrr     1.1.0     v tidyr     1.3.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x purrr::accumulate() masks foreach::accumulate()
## x dplyr::filter()     masks stats::filter()
## x dplyr::lag()        masks stats::lag()
## x purrr::when()       masks foreach::when()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Cargando paquete requerido: palmerpenguins
##
##
## Adjuntando el paquete: 'palmerpenguins'
##
##
## The following objects are masked from 'package:datasets':
##
##     penguins, penguins_raw
##
##
## Cargando paquete requerido: missForest
##
## Cargando paquete requerido: mice
##
##
## Adjuntando el paquete: 'mice'
##
##
## The following object is masked from 'package:stats':
##
##     filter
##
##
## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

```
## [[1]]
## [1] TRUE
##
## [[2]]
## [1] TRUE
##
## [[3]]
## [1] TRUE
##
## [[4]]
## [1] TRUE
##
## [[5]]
## [1] TRUE
##
## [[6]]
## [1] TRUE
##
## [[7]]
## [1] TRUE
##
## [[8]]
## [1] TRUE
##
## [[9]]
```

```
## [1] TRUE
##
## [[10]]
## [1] TRUE
##
## [[11]]
## [1] TRUE
##
## [[12]]
## [1] TRUE
##
## [[13]]
## [1] TRUE
##
## [[14]]
## [1] TRUE
##
## [[15]]
## [1] TRUE
```

```
rm(list.of.packages, new.packages)
```

The dataset will be simplified a bit.

```
data("penguins")
?penguins
```

```
## starting httpd help server ... done
```

```
cols <- c("species", "bill_length_mm", "bill_depth_mm", "flipper_length_mm",
          "body_mass_g", "sex")
df <- penguins %>%
  select(cols)
```

```
## Warning: Using an external vector in selections was deprecated in tidyselect 1.1.0.
## i Please use ‘all_of()‘ or ‘any_of()‘ instead.
##   # Was:
##   data %>% select(cols)
##
##   # Now:
##   data %>% select(all_of(cols))
##
## See <https://tidyselect.r-lib.org/reference/faq-external-vector.html>.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

```
df_clean <- df %>%
  select(-species) %>%
  drop_na()

n <- nrow(df_clean) # Number of observations
p <- ncol(df_clean) # Number of variables
```

Encoding

Q1: Transform the *sex* variable so it can be used in the analysis.

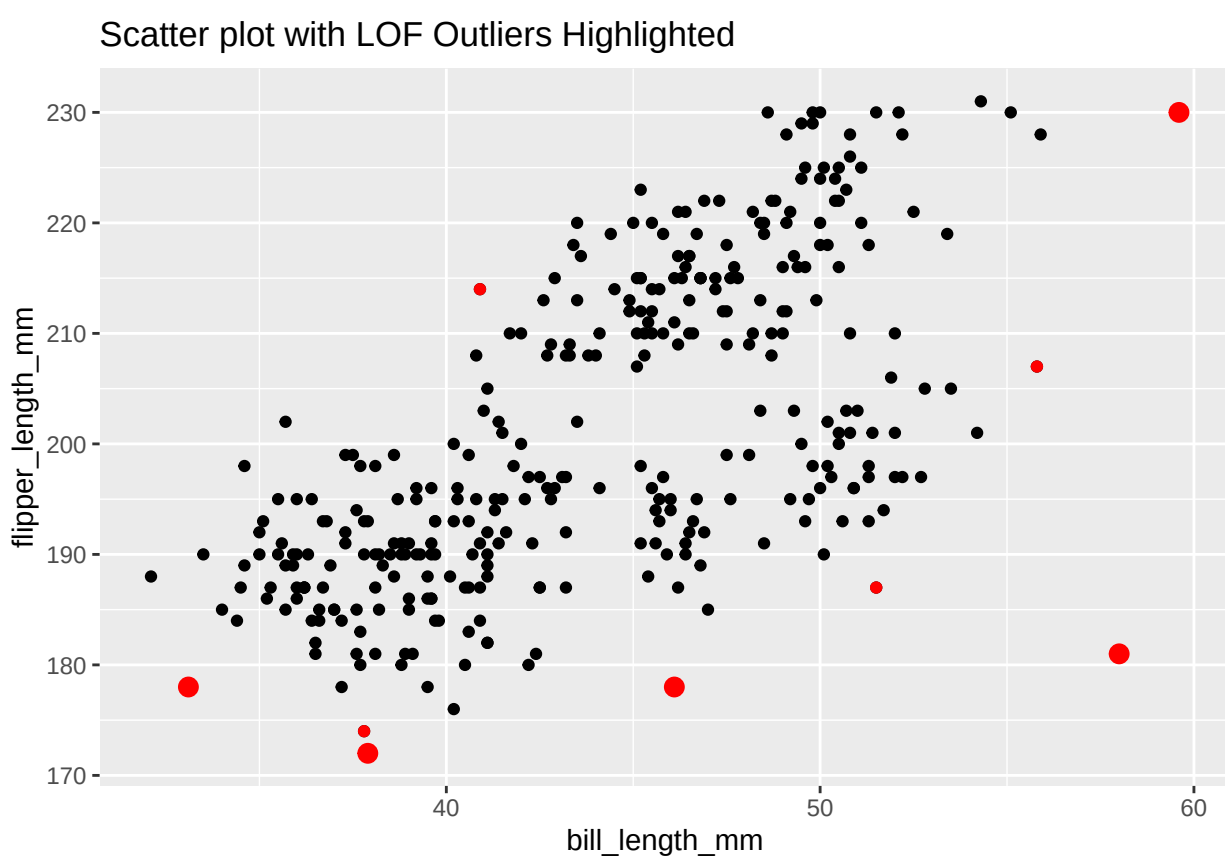
```
# transform de the sex variable so it can be used int he analysis
df_clean <- df_clean %>%
  mutate(sex = ifelse(sex == "male", 1, 0))
```

Outliers analysis

Using the **df_clean** dataframe answer the following questions. Focus on the variables *Flipper length* and *Bill length* for all the analyses.

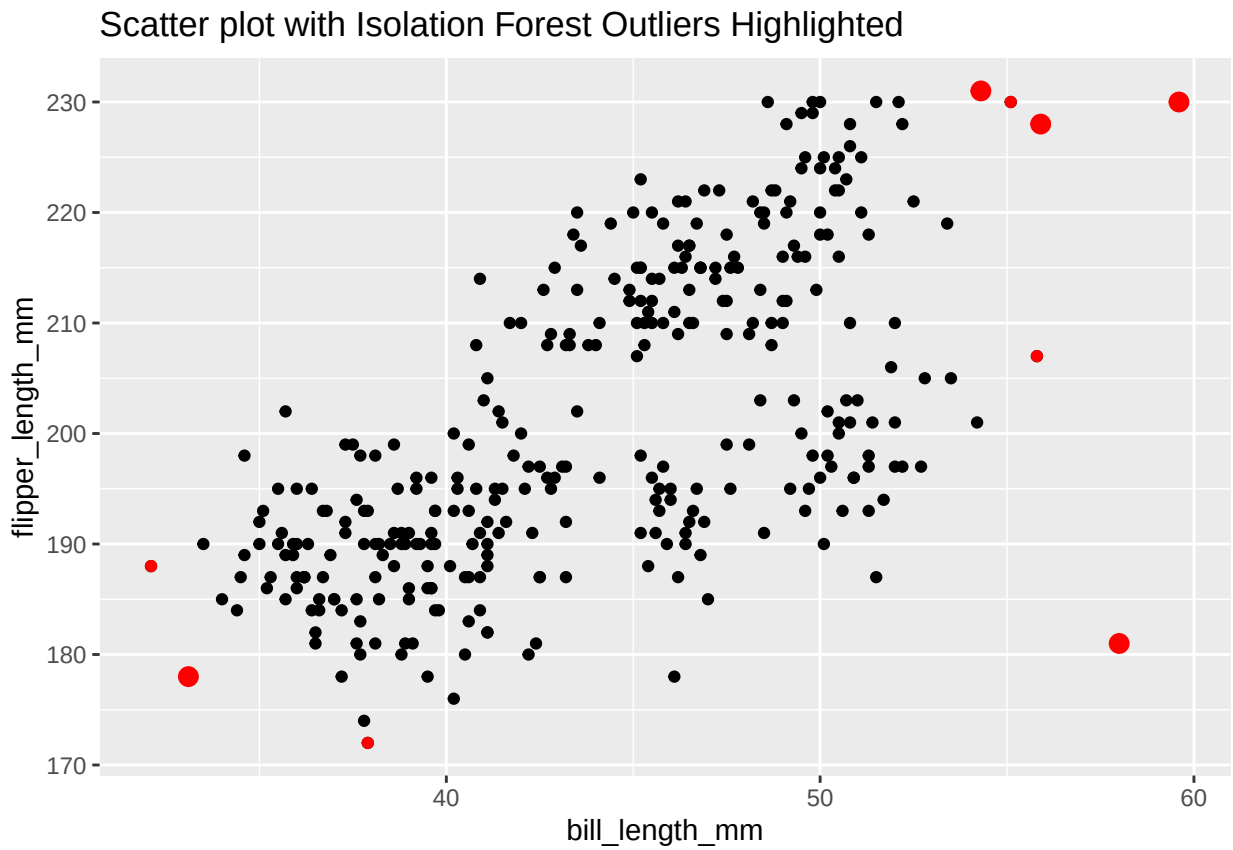
Q2: Use the Local Outlier Factor method (with $k = 20$) and identify the top five outliers using the quantile 97.5% tp find the score threshold. Generate a scatter plot to display the dataset, highlighting any observations classified as outliers.**

```
lof_scores <- lof(df_clean %>% select(bill_length_mm, flipper_length_mm), k = 20)
threshold <- quantile(lof_scores, 0.975)
outliers <- which(lof_scores > threshold)
top5 <- order(lof_scores, decreasing = TRUE)[1:5]
ggplot(df_clean, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_clean[outliers, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "red"
  geom_point(data = df_clean[top5, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "red", si
  labs(title = "Scatter plot with LOF Outliers Highlighted")
```



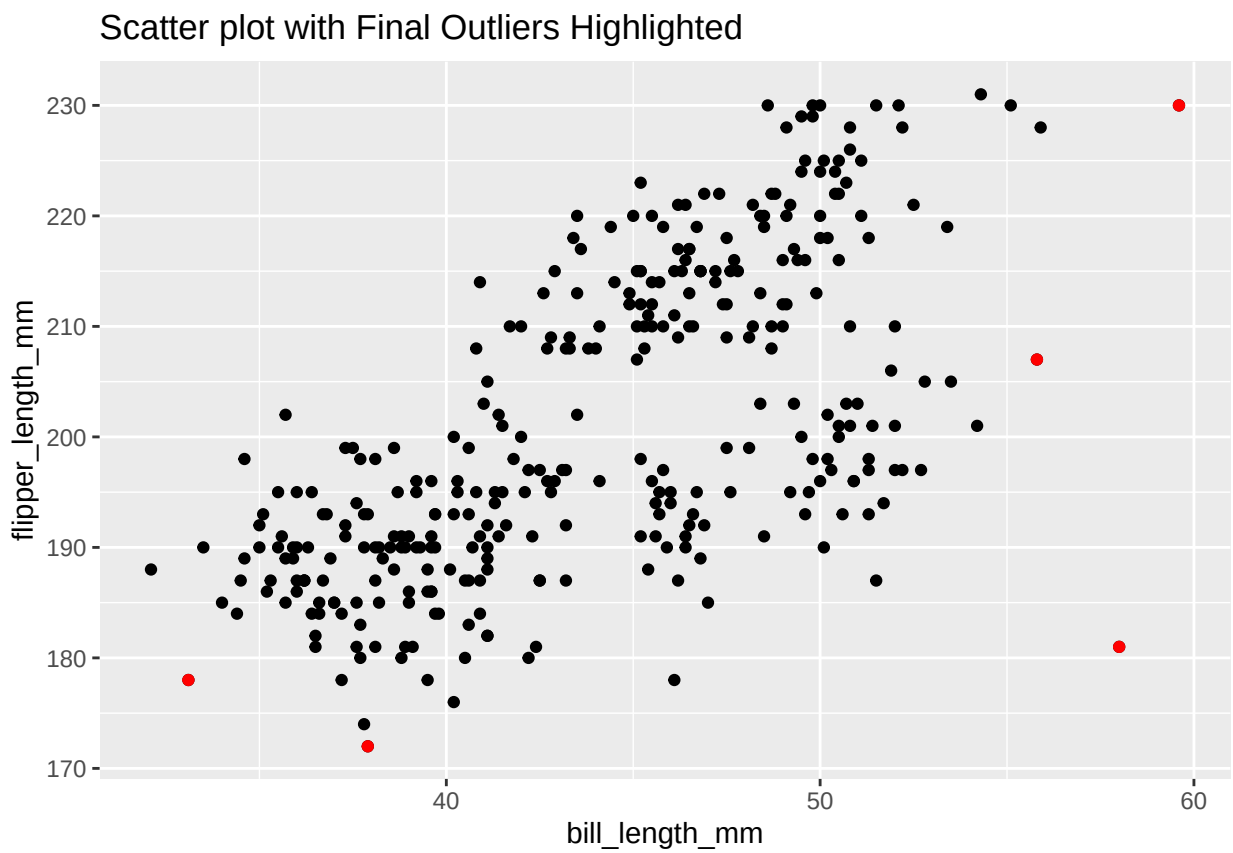
Q3: Use the Isolation Tree algorithm with 100 **trees and identify the top five outliers. Use the quantile** 97.5% **to find the score threshold** Generate a scatter plot to display the dataset, highlighting any observations classified as outliers.**

```
iso_model <- isolationForest$new(num_trees = 100)
iso_model$fit(df_clean %>% select(bill_length_mm, flipper_length_mm))
iso_scores <- iso_model$predict(df_clean %>% select(bill_length_mm, flipper_length_mm))$anomaly_score
threshold_iso <- quantile(iso_scores, 0.975)
outliers_iso <- which(iso_scores > threshold_iso)
top5 <- order(iso_scores, decreasing = TRUE)[1:5]
ggplot(df_clean, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_clean[outliers_iso, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "
  geom_point(data = df_clean[top5, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "red", si
  labs(title = "Scatter plot with Isolation Forest Outliers Highlighted")
```



Q4: From the previous results, which observations can be considered outliers? Why? Discuss what actions should be taken regarding them

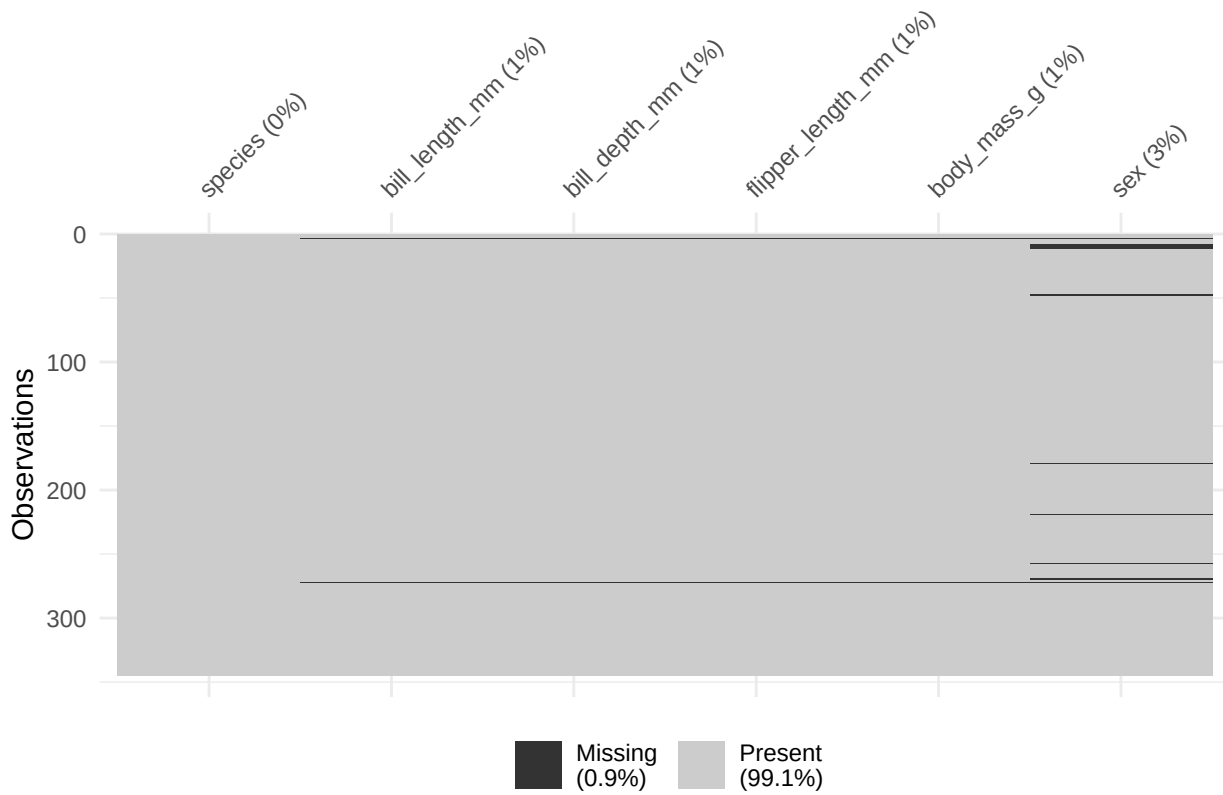
```
# The observations that can be considered outliers are those that have been identified by both methods
outliers_final <- intersect(outliers, outliers_iso)
# These observations are considered outliers because they have high scores in both methods. The measure
# plot the points of the intersection
ggplot(df_clean, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_clean[outliers_final, ], aes(x = bill_length_mm, y = flipper_length_mm), color =
  labs(title = "Scatter plot with Final Outliers Highlighted")
```



Missing values analysis Now using the `df` dataframe, answer the following questions:

Q5: Are there missing values? How we classify them (MCAR; MAR; MNAR)? Quantify the quality of the data using the missing values.

```
library(naniar)
vis_miss(df)
```




```r
paste0("Percentage of complete cases: ", round(sum(complete.cases(df))/nrow(df)*100,1), "%")
```

```
## [1] "Percentage of complete cases: 96.8%"
```

Yes, there are two types of missing values. There are some observations where absolutely no information exists (all columns are null except the spcecies column) and there are some observations where the column sex is missing. The first type of missing values can be classified as MCAR, since the fact that they are missing is completely random and does not depend on any other variable. The second type of missing values can be classified as MAR, the missingness is related to observed data but not the missing values themselves.

The data set presents an excellent quality as we consider that there's no MNAR and the percentge of missing values is 0.9% and the percentage of complete cases is 96.8%.

Q6: Perform a k-NN Imputation and a MICE imputation. Compare them. Which imputation has better quality? Why?

Hint: I provide code for the plots. Store the k-NN imputation in a dataframe with name `df_1nn` and the MICE imputation in a dataframe with name `df_mice` in order to perform the plots. Think carefully your answers and check the consistency of the imputations.

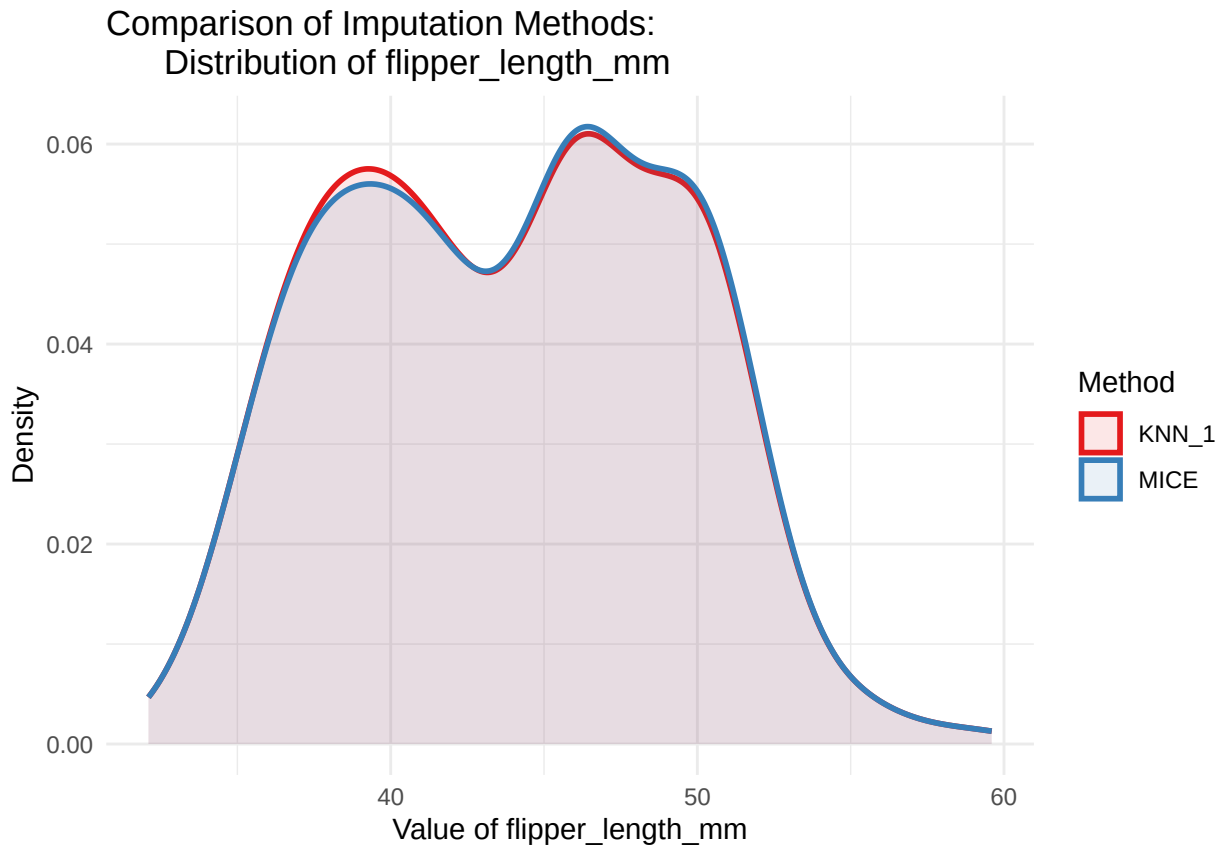
```r
#store the k-NN imputation in a dataframe with name df_1nn and the MICE imputation in a dataframe with
# k-NN imputation
set.seed(123)
df_1nn <- knnImputation(df %>% select(-species), k = 1)

# MICE imputation
```

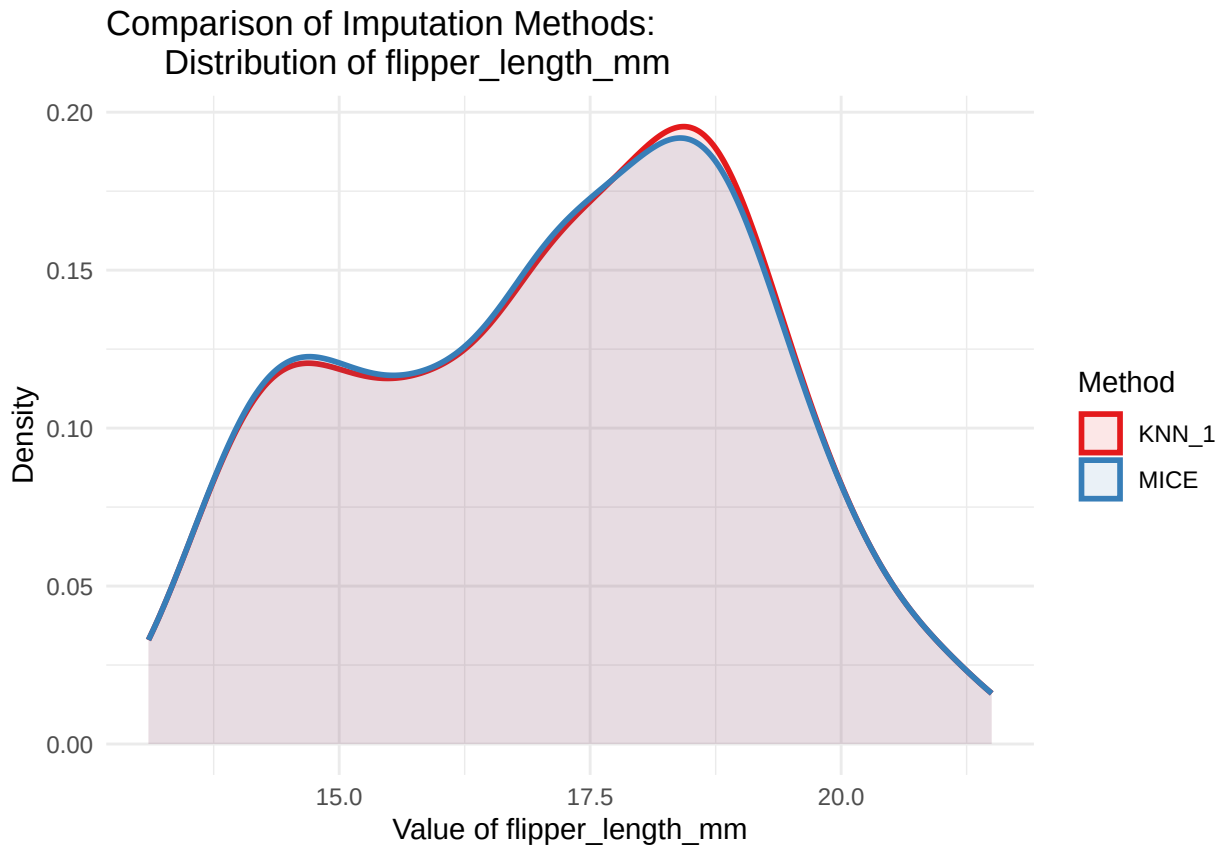
```r
set.seed(123)
mice_mod <- mice(df %>% select(-species), m = 1, method = "pmm", maxit = 5)
```

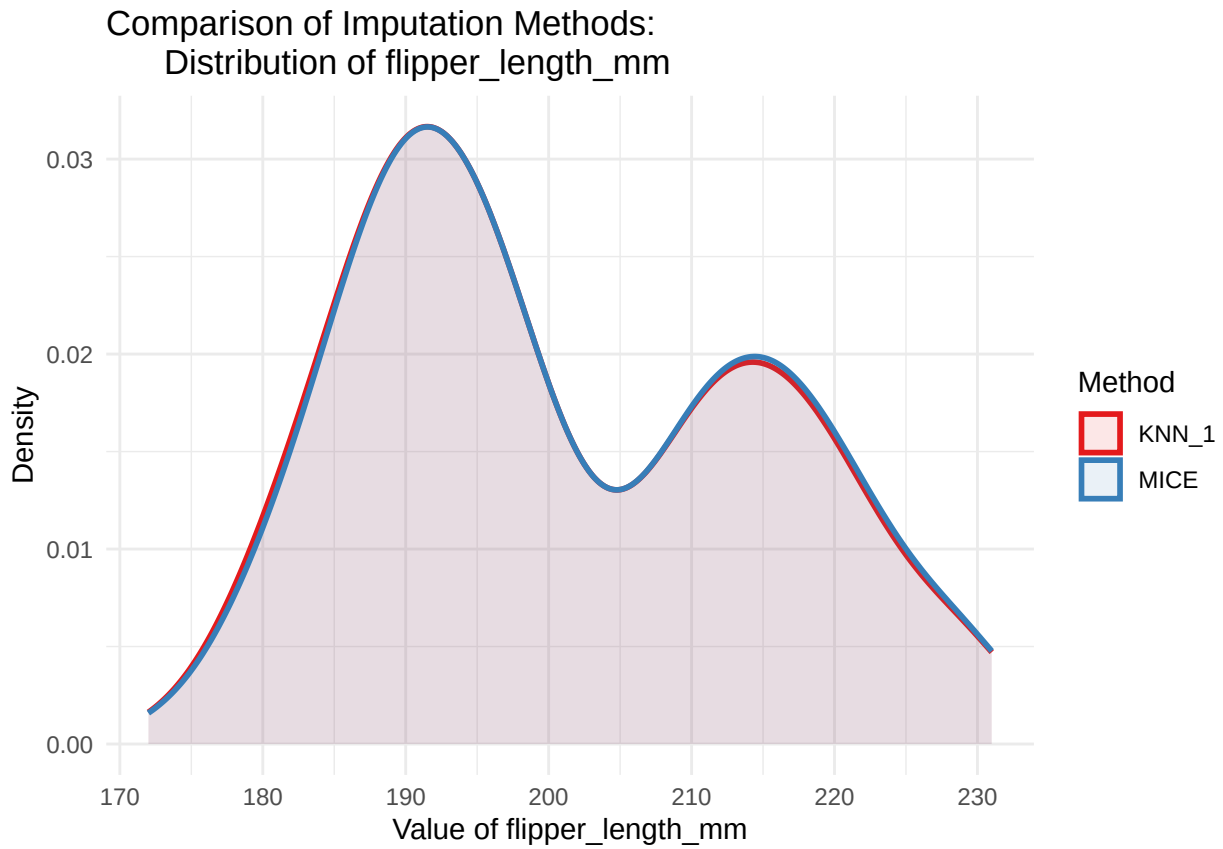
```
##
##  iter imp variable
##   1   1  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  sex
##   2   1  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  sex
##   3   1  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  sex
##   4   1  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  sex
##   5   1  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  sex
```

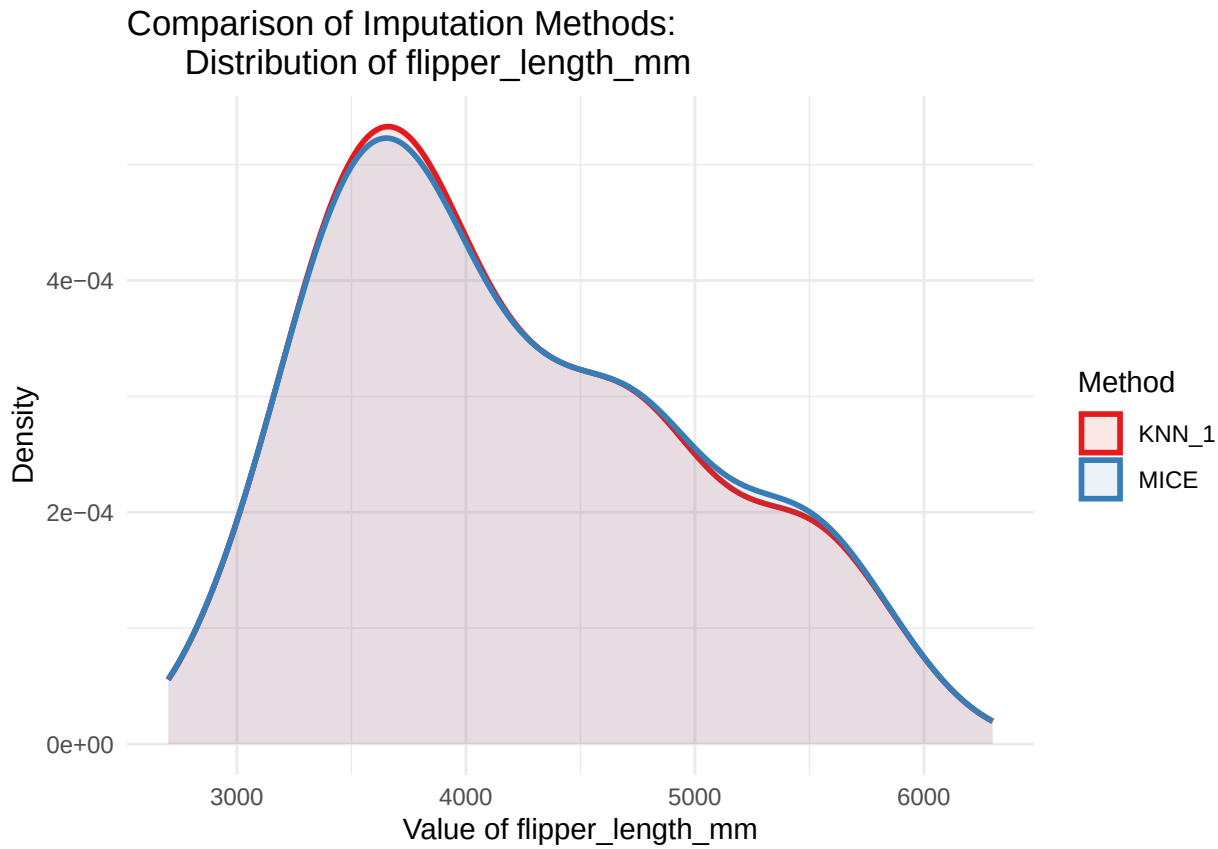
```r
df_mice <- complete(mice_mod)
```

```r
plot_columns <- cols[-c(1,6)]

for (column in plot_columns){
  comparison_df <- data.frame(
  KNN_1    = df_1nn[[column]],
  MICE     = df_mice[[column]]
  )
  plot_data <- comparison_df %>%
    pivot_longer(cols = everything(), names_to = "Method", values_to = "Value")

  p <- ggplot(plot_data, aes(x = Value, color = Method, fill = Method)) +
    geom_density(alpha = 0.1, linewidth = 1) +
    labs(
      title = "Comparison of Imputation Methods:
      Distribution of flipper_length_mm",
      x = "Value of flipper_length_mm",
      y = "Density"
    ) +
    theme_minimal() +
    scale_color_brewer(palette = "Set1") +
    scale_fill_brewer(palette = "Set1")

  print(p)

}
```


Comparison of Imputation Methods:
Distribution of flipper_length_mm
0.06
0.04
0.02
0.00
Density
40
50
60
Value of flipper_length_mm
Method
KNN_1
MICE


Comparison of Imputation Methods:
Distribution of flipper_length_mm
0.20
0.15
0.10
0.05
0.00
Density
15.0
17.5
20.0
Value of flipper_length_mm
Method
KNN_1
MICE

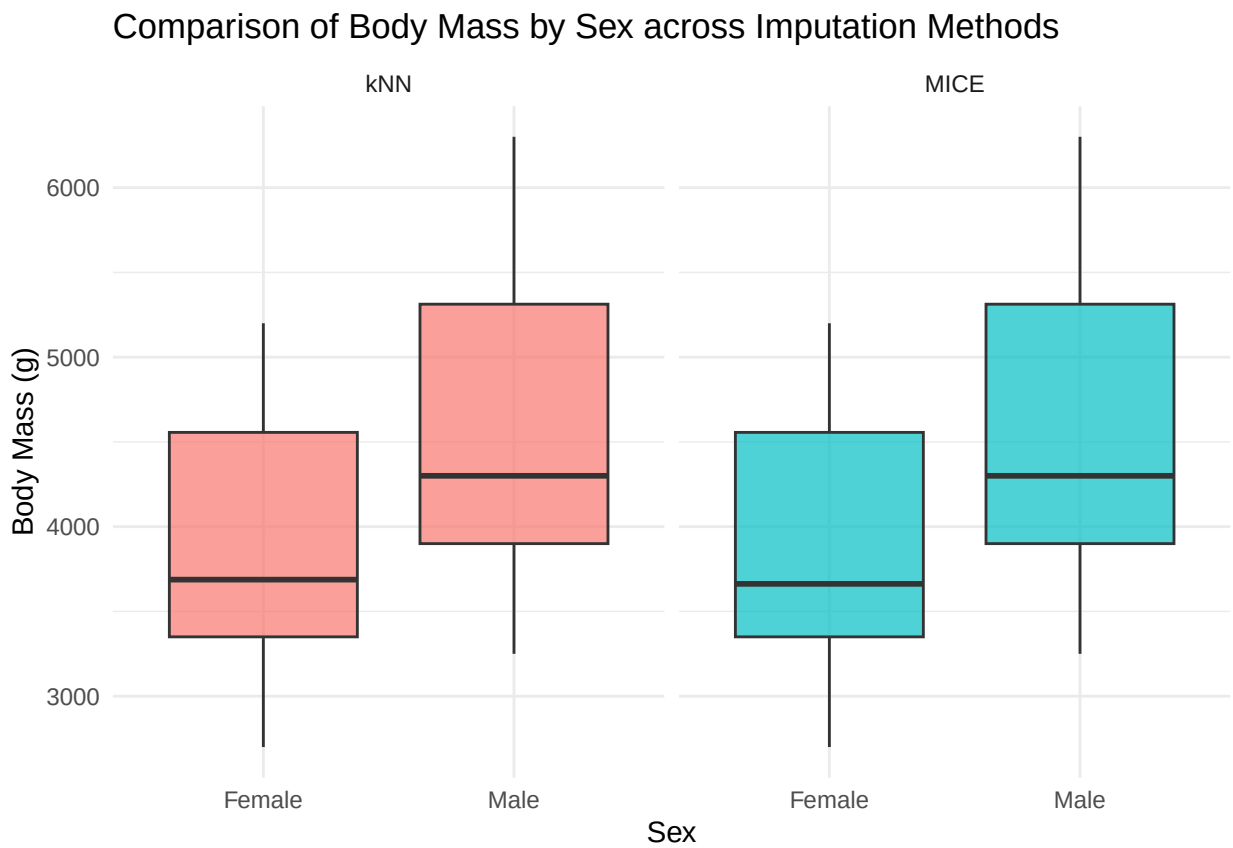

Comparison of Imputation Methods:
Distribution of flipper_length_mm
Density
0.03
0.02
0.01
0.00
170
180
190
200
210
220
230
Value of flipper_length_mm
Method
KNN_1
MICE



MICE seems to give slightly better results than KNN because its curves are a bit closer to the original distributions in all the plots. With KNN, the distributions look a little more smoothed, which suggests it is averaging values and slightly reducing the variability and the extremes. MICE keeps the peaks, spread and tails a bit better, so it preserves the overall shape of the data more naturally. Both methods work quite similarly, but overall MICE looks like the more reliable option.

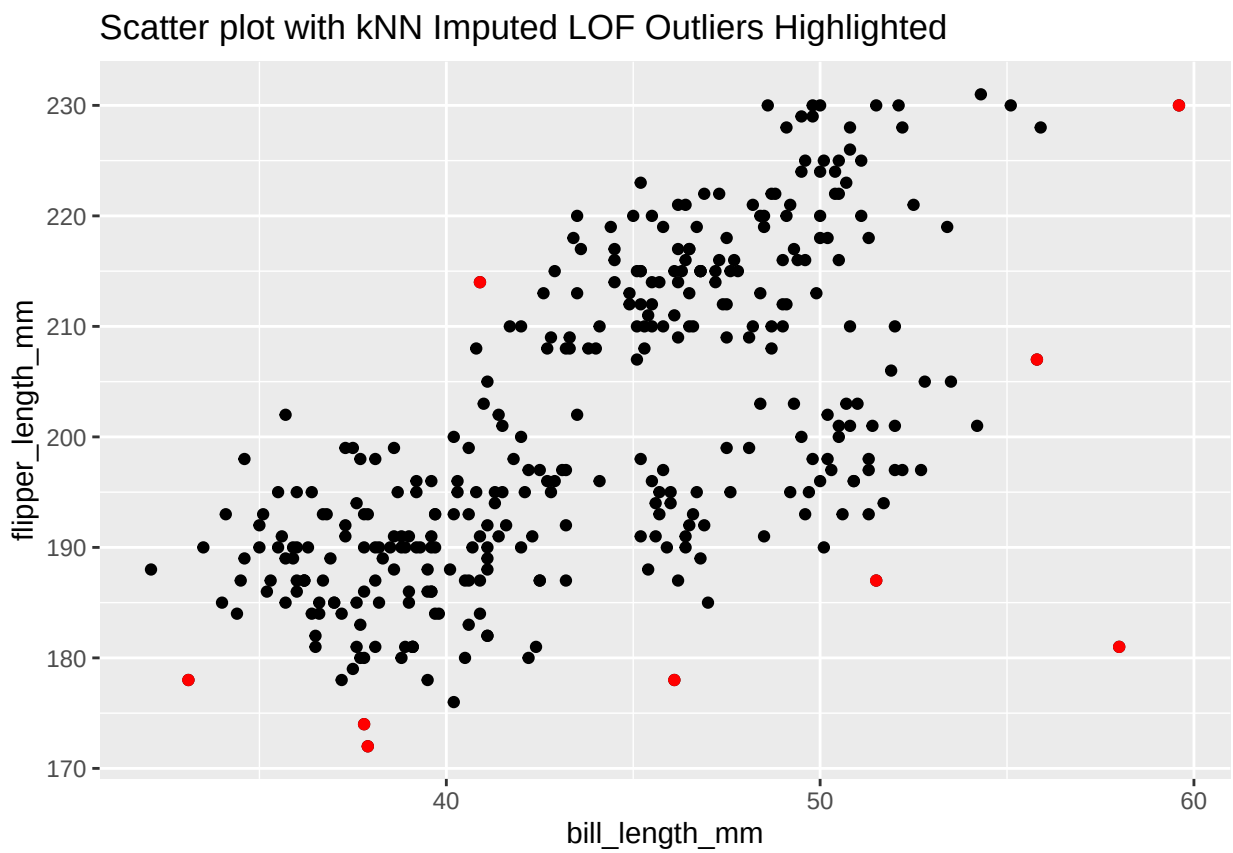
```
subset_knn <- df_1nn %>%
  select(body_mass_g, sex) %>%
  mutate(Method = "kNN")

subset_mice <- df_mice %>%
  select(body_mass_g, sex) %>%
  mutate(Method = "MICE")

df_comparison <- bind_rows(subset_knn, subset_mice)

df_comparison <- df_comparison %>%
  mutate(sex = factor(sex, labels = c("Female", "Male")))

ggplot(df_comparison, aes(x = sex, y = body_mass_g, fill = Method)) +
  geom_boxplot(alpha = 0.7) +
  facet_wrap(~ Method) +
  labs(
    title = "Comparison of Body Mass by Sex across Imputation Methods",
    x = "Sex",
    y = "Body Mass (g)"
  ) +
```

```
  theme_minimal() +
  theme(legend.position = "none")
```

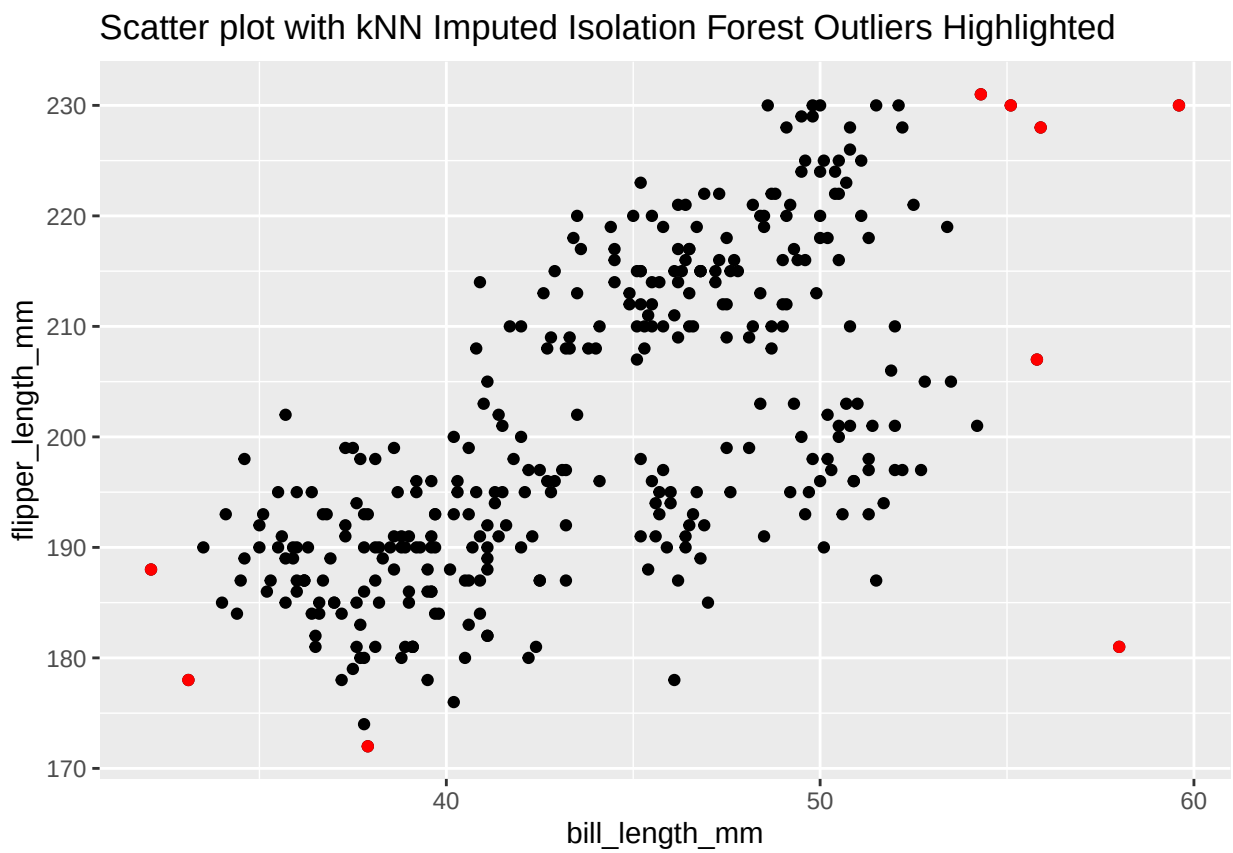


The other way **Q7: Using the data frame with imputed data (KNN and MICE), repeat the outlier analysis (steps Q1-Q4). Compare the results of the two approaches: performing outlier analysis on a complete-case dataset (removing NAs) versus imputing missing values before conducting the outlier analysis.**

```
# kNN data
lof_scores_knn <- lof(df_1nn %>% select(bill_length_mm, flipper_length_mm), k = 20)
threshold_knn <- quantile(lof_scores_knn, 0.975)
outliers_knn <- which(lof_scores_knn > threshold_knn)
ggplot(df_1nn, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_1nn[outliers_knn, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "re
  labs(title = "Scatter plot with kNN Imputed LOF Outliers Highlighted")
```



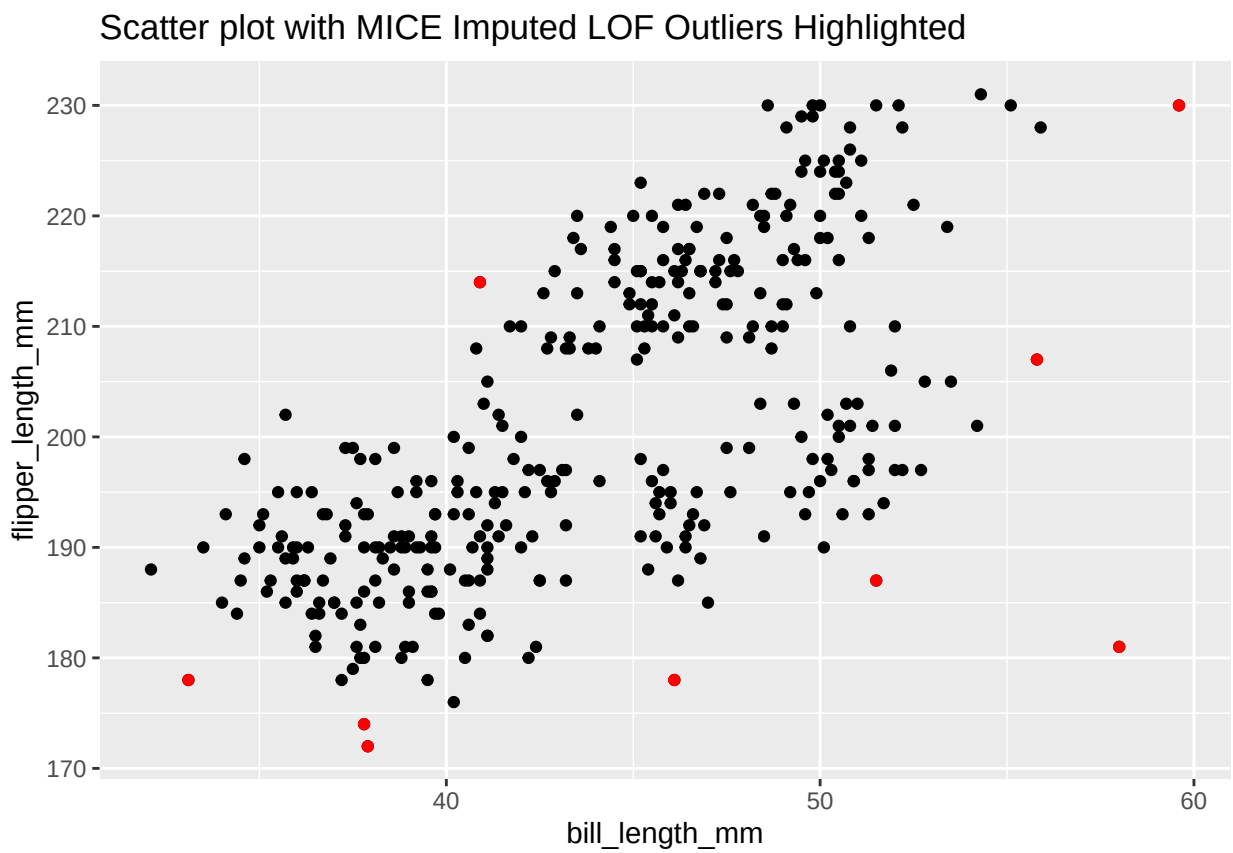

```
# also the isolation tree algorithm
iso_model_knn <- isolationForest$new(num_trees = 100)
iso_model_knn$fit(df_1nn %>% select(bill_length_mm, flipper_length_mm))
iso_scores_knn <- iso_model_knn$predict(df_1nn %>% select(bill_length_mm, flipper_length_mm))$anomaly_s
threshold_iso_knn <- quantile(iso_scores_knn, 0.975)
outliers_iso_knn <- which(iso_scores_knn > threshold_iso_knn)
ggplot(df_1nn, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_1nn[outliers_iso_knn, ], aes(x = bill_length_mm, y = flipper_length_mm), color =
  labs(title = "Scatter plot with kNN Imputed Isolation Forest Outliers Highlighted")
```



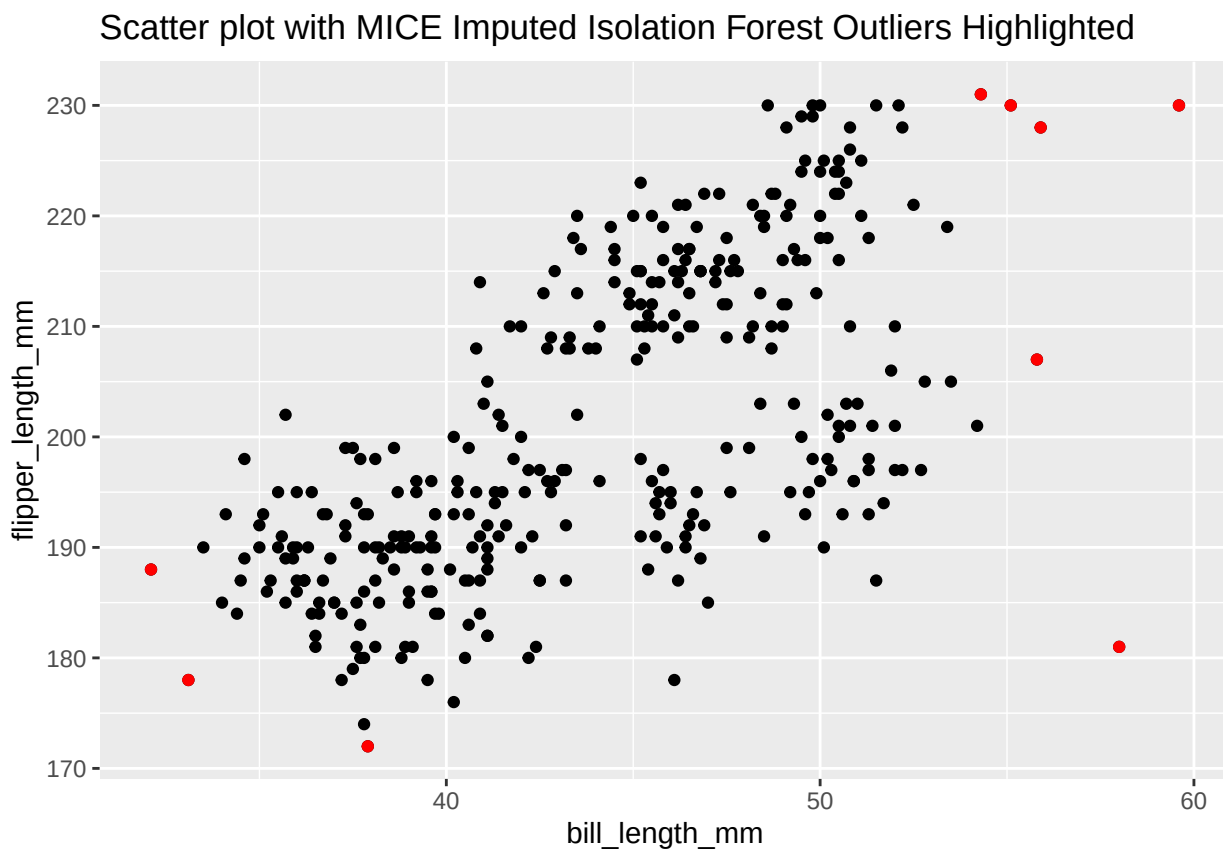
Because they are clearly separated from the bulk of the data, they can be considered true anomalies. They should be investigated for possible data errors, but for modelling and analysis it would be reasonable to remove them, or at least compare results with and without them, since they could distort statistical results and model performance.

Now we perform the same prodecure with the MICE imputed data:

```
# MICE data
lof_scores_mice <- lof(df_mice %>% select(bill_length_mm, flipper_length_mm), k = 20)
threshold_mice <- quantile(lof_scores_mice, 0.975)
outliers_mice <- which(lof_scores_mice > threshold_mice)
ggplot(df_mice, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_mice[outliers_mice, ], aes(x = bill_length_mm, y = flipper_length_mm), color = "
  labs(title = "Scatter plot with MICE Imputed LOF Outliers Highlighted")
```




```
# also the isolation tree algorithm
iso_model_mice <- isolationForest$new(num_trees = 100)
iso_model_mice$fit(df_mice %>% select(bill_length_mm, flipper_length_mm))
iso_scores_mice <- iso_model_mice$predict(df_mice %>% select(bill_length_mm, flipper_length_mm))$anomaly
threshold_iso_mice <- quantile(iso_scores_mice, 0.975)
outliers_iso_mice <- which(iso_scores_mice > threshold_iso_mice)
ggplot(df_mice, aes(x = bill_length_mm, y = flipper_length_mm)) +
  geom_point() +
  geom_point(data = df_mice[outliers_iso_mice, ], aes(x = bill_length_mm, y = flipper_length_mm), color
  labs(title = "Scatter plot with MICE Imputed Isolation Forest Outliers Highlighted")
```



I apply a similar reasoning as before.

When comparing both approaches, the overall pattern of outliers is quite similar, but the imputation approach gives more stable and reliable results. With the complete-case dataset, many observations are removed due to missing values, which reduces the sample size and can change the data distribution, so the detected outliers depend on a smaller and potentially biased subset of the data. After imputing the missing values, we keep all observations, the structure of the dataset is preserved, and the Isolation Forest detects outliers in a more consistent way. In practice, this means that performing imputation before outlier detection is preferable, because it uses more information and avoids losing potentially important observations.

Multivariate matrices

Using the data without missing values (complete cases: `df_clean`) answer the following questions:

Q8: Compute the sample mean vector m, the centring matrix H and the centered data matrix X_c. Only matrix and vector operations are allowed.

```
# Sample mean vector
n <- nrow(df_clean)
m <- t(as.matrix(df_clean)) %*% matrix(1, n, 1) / n
# print
print(m)
```

```
##                          [,1]
## bill_length_mm      43.9927928
## bill_depth_mm       17.1648649
```

```
## flipper_length_mm  200.9669670
## body_mass_g       4207.0570571
## sex                  0.5045045
```

```r
# Centering matrix
H <- diag(n) - (1/n) * matrix(1, n, n)
#print(H)

# Centered data matrix
X_c <- H %*% as.matrix(df_clean)
#print(X_c)
```

Q9: Using the centered data matrix compute the sample covariance matrix S. Only matrix and vector operations are allowed.

```r
S <- (1/(n-1)) * t(X_c) %*% X_c
print(S)
```

```
##                   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
## bill_length_mm        29.9063334     -2.462091         50.058195   2595.6233
## bill_depth_mm         -2.4620913      3.877888        -15.947248   -748.4561
## flipper_length_mm     50.0581949    -15.947248        196.441677   9852.1916
## body_mass_g         2595.6233040   -748.456122       9852.191649 648372.4877
## sex                    0.9422012      0.367478          1.790812    171.3537
##                           sex
## bill_length_mm      0.9422012
## bill_depth_mm       0.3674780
## flipper_length_mm   1.7908119
## body_mass_g       171.3536579
## sex                 0.2507327
```

Q10: Obtain the standardized data matrix X_s and use it to compute the sample correlation matrix R. Only matrix and vector operations are allowed.

```r
# Standardized data matrix
X_s <- X_c %*% diag(1/sqrt(diag(S)))
#print(X_s)
# Sample correlation matrix
R <- (1/(n-1)) * t(X_s) %*% X_s
print(R)
```

```
##            [,1]       [,2]       [,3]       [,4]      [,5]
## [1,]  1.0000000 -0.2286256  0.6530956  0.5894511 0.3440778
## [2,] -0.2286256  1.0000000 -0.5777917 -0.4720157 0.3726733
## [3,]  0.6530956 -0.5777917  1.0000000  0.8729789 0.2551689
## [4,]  0.5894511 -0.4720157  0.8729789  1.0000000 0.4249870
## [5,]  0.3440778  0.3726733  0.2551689  0.4249870 1.0000000
```